

# Real-world (build and search a real knowledge base) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

**Story — `knowledge_base_search_practice.py`** : writing every document yourself means you're your own ground truth — when a search comes back wrong, you know it's wrong immediately, instead of having to guess whether the retrieval or the answer is at fault. **If not:** the Build Task's knowledge base would be the first place you ever had to debug a retrieval bug without knowing what the right answer was supposed to be.

Every example below assumes the same 6 sample documents ( `docs/*.txt` ):

```
# knowledge_base_search_practice.py
DOCS: dict[str, str] = {
    "French press basics.txt": (
        "A French press steeps coarse coffee grounds directly in hot "
        "water, then a mesh plunger separates the grounds. Steep for "
        "about 4 minutes before pressing. It produces a fuller-bodied, "
        "slightly gritty cup compared to filtered methods."
    ),
    "Pour-over basics.txt": (
        "Pour-over brewing means pouring hot water over grounds in a "
        "paper filter, in slow circular motions, letting it drip through "
        "into a cup or carafe below. It produces a clean, light-bodied "
        "cup because the paper filter traps most oils and fine particles."
    ),
    "Grind size guide.txt": (
        "Grind size should match your brew method. Use a coarse grind "
        "for French press, a medium grind for pour-over and drip "
        "machines, and a fine grind for espresso. Too fine a grind for "
        "the method causes over-extraction and bitterness."
    ),
    "Water temperature.txt": (
        "Water for brewing coffee should be between 195 and 205 degrees "
        "Fahrenheit, just off a full boil. Water that's too cool "
        "under-extracts and tastes sour or weak. Water that's too hot "
        "can scald the grounds and taste bitter."
    ),
    "Coffee-to-water ratio.txt": (
        "A common starting ratio is 1 gram of coffee to 16 grams of "
```

```

        "water, sometimes written as 1:16. For a stronger cup, move "
        "toward 1:14. For a milder cup, move toward 1:18. Adjust from "
        "there based on taste."
    ),
    "Storing coffee beans.txt": (
        "Store coffee beans in an airtight container, away from light "
        "and heat. Whole beans stay fresh for a few weeks after "
        "roasting; ground coffee stales much faster, within days. Don't "
        "store beans in the freezer for everyday use, since condensation "
        "on thawing hurts flavor."
    ),
}

```

## Basic Version

---

### Approach 1 — raw `chromadb` client

```

# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

docs_folder = Path("docs")
docs_folder.mkdir(exist_ok=True)
for filename, text in DOCS.items():
    (docs_folder / filename).write_text(text)

def embed(text):
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding

def build_vector_store(doc_folder):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="kb")
    for file_path in Path(doc_folder).iterdir():
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection

```

```

collection = build_vector_store("docs")

questions = [
    "What grind size should I use for a French press?",
    "How hot should the water be when brewing?",
    "How long does ground coffee stay fresh?",
]

for question in questions:
    question_embedding = embed(question)
    results = collection.query(
        query_embeddings=[question_embedding], n_results=3,
    )
    print("Q: " + question)
    print(results["ids"][0])
    print(results["documents"][0])
    print()

```

**Expected output:** since you wrote every doc yourself, question 1 should return `French press basics.txt` (and probably `Grind size guide.txt`) at the top, question 2 should return `Water temperature.txt`, and question 3 should return `Storing coffee beans.txt`. Read the printed results and check.

This version works correctly and meets every requirement of the exercise. Running the script a second time crashes, though — `create_collection` raises if `"kb"` already exists.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — raw `chromadb`, factored into functions with type hints

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

def write_sample_docs(doc_folder: str, docs: dict[str, str]) -> None:
    folder_path = Path(doc_folder)
    folder_path.mkdir(exist_ok=True)
    for filename, text in docs.items():
        (folder_path / filename).write_text(text)

def embed(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding

def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="kb")
    for file_path in Path(doc_folder).iterdir():
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection

def ask(collection, question: str) -> None:
    question_embedding = embed(question)
    results = collection.query(query_embeddings=[question_embedding], n_results=3)
```

```

print(f"Q: {question}")
print(results["ids"][0])
print(results["documents"][0])
print()

def main() -> None:
    write_sample_docs("docs", DOCS)
    collection = build_vector_store("docs")
    questions = [
        "What grind size should I use for a French press?",
        "How hot should the water be when brewing?",
        "How long does ground coffee stay fresh?",
    ]
    for question in questions:
        ask(collection, question)

if __name__ == "__main__":
    main()

```

## Approach 2 — LangChain's `Chroma` wrapper

```

# knowledge_base_search_practice.py
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

def build_vector_store(docs: dict[str, str]) -> Chroma:
    texts = list(docs.values())
    metadatas = [{"source": filename} for filename in docs.keys()]
    return Chroma.from_texts(texts, OpenAIEmbeddings(), metadatas=metadatas)

def ask(vector_store: Chroma, question: str) -> None:
    results = vector_store.similarity_search(question, k=3)
    print(f"Q: {question}")
    for result in results:
        print(result.metadata["source"], "- ", result.page_content)
    print()

def main() -> None:
    vector_store = build_vector_store(DOCS)
    questions = [
        "What grind size should I use for a French press?",

```

```

        "How hot should the water be when brewing?",
        "How long does ground coffee stay fresh?",
    ]
    for question in questions:
        ask(vector_store, question)

if __name__ == "__main__":
    main()

```

**Expected output:** same 3 correct top matches as Basic. `Chroma.from_texts` embeds every doc for you, and `similarity_search` embeds the question for you — no manual `embeddings.create()` call anywhere.

**Difference from Basic:** full type hints, and everything split into named functions ( `write_sample_docs` , `build_vector_store` , `ask` , `main` ) instead of one flat script — each function does one job and can be tested or reused on its own. Approach 2 additionally removes the manual embedding calls entirely by letting LangChain's `Chroma` wrapper handle them. Neither approach yet survives being run a second time, or tells you when a question's best match genuinely isn't relevant — that's what the next 2 approaches add.

### Approach 3 — `get_or_create_collection` , safe to call more than once

**Story:** `create_collection(name="kb")` raises if `"kb"` already exists — run this script a second time, for any reason, and it crashes on line 2 before doing anything useful. **If not:** the Build Task's ingest step would be a one-shot script instead of something safe to re-run after adding or editing a document, which is the normal case for a real knowledge base.

```

# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

def embed(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding

def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()

```

```

collection = chroma_client.get_or_create_collection(name="kb")

file_paths = list(Path(doc_folder).iterdir())
if collection.count() >= len(file_paths):
    # why: already built by an earlier call -- don't re-embed everything
    return collection

for file_path in file_paths:
    text = file_path.read_text()
    embedding = embed(text)
    collection.add(
        documents=[text], embeddings=[embedding], ids=[file_path.name],
    )
return collection

# proof this is now safe to call twice
first = build_vector_store("docs")
second = build_vector_store("docs")
print(first.count(), second.count())

```

**Expected output:**

```
6 6
```

The second call doesn't crash, and doesn't double-add every document either —

`collection.count() >= len(file_paths)` short-circuits it.

**Approach 4 — a score cutoff, so "no relevant match" is a real, checked outcome**

**Story:** a nearest-neighbor search always returns *something* — ask an off-topic question and `collection.query()` still confidently hands back its `n_results` closest matches, with nothing signaling "none of these are actually good." **If not:** the Build Task's agent would silently receive 3 irrelevant chunks for an off-topic question and guess an answer from them, instead of being able to say "I don't have information on that."

```

# knowledge_base_search_practice.py
SCORE_CUTOFF: float = 0.35 # tune against your own embedding model + documents

def search(
    collection, question: str, k: int = 3, cutoff: float = SCORE_CUTOFF,
) -> list[dict]:
    question_embedding = embed(question)
    results = collection.query(query_embeddings=[question_embedding], n_results=k

```

```

matches: list[dict] = []
for doc_id, text, distance in zip(
    results["ids"][0], results["documents"][0], results["distances"][0]
):
    if distance <= cutoff:
        matches.append({"id": doc_id, "text": text, "score": distance})

return matches    # empty list means "nothing relevant," not "search failed"

collection = build_vector_store("docs")

good_question = "What grind size should I use for a French press?"
off_topic_question = "What's the best way to train a marathon?"

print("on-topic:", search(collection, good_question))
print("off-topic:", search(collection, off_topic_question))

```

**Expected output** (exact scores vary, but the shape holds):

```

on-topic: [{'id': 'French press basics.txt', 'text': '...', 'score': 0.18}, ...]
off-topic: []

```

The on-topic question returns real matches under the cutoff. The off-topic question still gets a nearest neighbor internally — nearest-neighbor search always finds something — but every distance is worse than `cutoff`, so `search()` filters all of them out and returns `[]` instead of confidently handing back 3 irrelevant chunks.

### Difference from Approach 1/2, and between Approaches 3/4: Approach 1/2's

`build_vector_store` crashes on a second call, and its `ask / similarity_search` always return *something*, whether or not it's actually relevant. Approach 3 fixes the first problem with `get_or_create_collection` plus a cheap "already built" check — the same "check first, only do the real work if needed" pattern as Doc01's `load_config()` cache. Approach 4 fixes a different problem: it makes "nothing here is actually relevant" a real, testable outcome instead of a silent 3-result list that looks identical whether the match was great or terrible.

**Which one should you actually use?** Both, together, are what a real knowledge base needs — they solve genuinely different problems. `get_or_create_collection` (Approach 3) costs nothing and should just always be there, the moment there's any chance this code runs more than once. The score cutoff (Approach 4) needs real tuning against your own documents and embedding model — a cutoff picked without ever checking a genuinely off-topic question against it is just a guess. Both directly feed the Build Task below: `retrieve()`'s "handles an empty set of documents without crashing" requirement is the same idea as `get_or_create_collection`'s safety, and "off-topic

questions should return low scores... not confidently-wrong top results" is exactly what Approach 4's cutoff gives you.